Git & GitHub Cheat Sheet

A complete, scannable command reference — organized by workflow stage

⚙ 1. Initial setup & configuration

Configure your global identity so commits are stamped with your name and email.

```
git config --global user.name "Your Name"
```

Sets the global username for your commits.

```
git config --global user.email "yourmail@example.com"
```

Sets the global email address for your commits.

```
git config --global --list
```

Lists all active configurations to verify they were set correctly.

🧭 2. Terminal navigation essentials

Baseline shell commands for moving around and managing folders.

```
pwd
```

Print working directory — shows the absolute path of your current folder.

```
ls
```

Lists all visible files and folders in the current directory.

```
ls -a
```

Lists visible AND hidden files/folders — useful for seeing the hidden `.git` folder.

```
cd <folder_name>
```

Moves forward into the specified folder.

```
cd ..
```

Moves one step back to the parent directory.

```
mkdir <folder_name>
```

Creates a new empty folder in the current location.

```
clear
```

Clears the terminal screen.

🌐 3. Existing repo workflow (GitHub-first)

Use this when the repo already exists on GitHub and you're bringing it onto your machine.

```
git clone <repository_https_url>
```

Downloads a remote repository from GitHub onto your local machine.

```
git status
```
Shows the current state of files — untracked, modified, or staged.

> 💡 **The two-step commit process:**
> Think of it like a relationship milestone —
> 1. **Staging** (`git add`) → the "engagement" stage, files are prepared.
> 2. **Committing** (`git commit`) → the "wedding" stage, changes are permanently recorded.

```
git add <file_name>
```
Moves a specific file's changes into the staging area.

```
git add .
```
Stages all new, modified, and deleted files at once.

```
git commit -m "message"
```
Permanently saves staged changes to local history.

```
git push origin main
```
Sends local commits to the remote GitHub repository on the **`main`** branch.

📁 4. Local-first workflow (new repo)

Use this when you have an existing folder and want to turn it into a Git repo + push to GitHub.

```
git init
```
Initializes a new local repo by creating a hidden **`.git`** tracking folder.

```
git remote add origin <repository_https_url>
```
Connects your local repo to a remote one, naming it **`origin`**.

```
git remote -v
```
Verifies the connection by showing the linked remote URLs.

```
git push -u origin main
```
Pushes for the first time and sets upstream tracking — future pushes just need **`git push`**.

🌿 5. Branch management

Branches let you work on features in isolation without overwriting others' work.

```
git branch
```
Lists all local branches — current active branch is marked with *.

```
git branch -m <new_branch_name>
```
Renames the current branch (e.g. old **`master`** → **`main`**).

```
git checkout <branch_name>
```
Switches your active workspace to the specified branch.

```
git checkout -b <new_branch_name>
```
Creates a new branch and switches to it instantly.

Creates a new branch and switches to it instantly.

`git branch -d <branch_name>`

Deletes a local branch (switch away from it first).

`git push origin <branch_name>`

Pushes a feature branch to GitHub to share with the team.

🔀 6. Merging & collaborative syncing

Bring a finished feature back into the main timeline and stay in sync with teammates.

`git diff <branch_name>`

Compares your active branch with another to see what lines differ.

`git pull origin main`

Fetches and merges the latest remote changes into your local workspace.

> ⚠ **Resolving merge conflicts**
>
> 1. Open the file — look for `<<<<<<< HEAD`, `=======`, and `>>>>>>> branch_name` markers.
> 2. Use your editor's options to accept current, incoming, or both changes.
> 3. Save, then run `git add .` and `git commit -m "resolved merge conflict"`.

🕰 7. History & undoing changes

Track your history and roll back mistakes safely (or not so safely).

`git log`

Shows commit history with hash IDs. Press **q** to exit.

`git reset <file_name>`

Unstages a file, moving it back to a normal modified state.

`git reset HEAD~1`

Rolls back one commit — changes return to your unstaged working folder.

`git reset <commit_hash_id>`

Rolls history back to a specific past commit by its hash.

> 🚨 **Extreme caution:** `git reset --hard <commit_hash_id>` instantly destroys current progress and resets both history and files to match that commit exactly.

🌍 8. Open source & GitHub web features

Actions performed on github.com rather than the terminal.

🍴 Forking

Creates a full copy of someone else's public repo into your own account — your personal "rough notebook" to experiment with.

Pull request (PR)

A formal request telling a repo's owners that you've pushed changes and would like them reviewed and merged into the main project.